o-means: Two-Level Assignment for Mini-Batch k-Means at Large k

Rahul Bajaj, Jaya Nupur, Anuj Garg
Owlgebra AI

November 2026

Abstract

Mini-batch k-means with exponential-moving-average updates (e-means) matches the error of 25 Lloyd iterations in two to three passes over the data, but in every pass it still compares each point with all k centroids. We present o-means, which keeps e-means' training loop and replaces its assignment step. The centroid update is unchanged: the same exponential-moving-average rule, the same batch-size and step-size schedules, and the same revival of dead clusters. Assignment becomes a two-level search. The k centroids are grouped into $G \approx k/128$ cells; a point first finds its p nearest cells (p between 12 and 24) and is then compared only with the centroids inside them, which is 6–23% of the distance evaluations of an exhaustive pass. Because the centroids move at every step, the grouping cannot be built once and reused: o-means maintains the cells throughout training as the centroids drift and are split. Assignment during training is approximate and has no worst-case guarantee. Runs shorter than two epochs end with a recency-weighted mean. We evaluate under a protocol locked before any scored run, on eight held-out datasets (0.24–10 million vectors, 100–3,072 dimensions, $k = 8{,}192$–$65{,}536$) and one RTX 5090, where e-means runs its Triton backend rather than its H100 kernels. All errors are scored with exact assignment on held-out rows. On every dataset o-means reaches the error of e-means' 1-, 2- and 3-epoch runs in less time: 4.49× faster in geometric mean (4.08× counting only measured budgets) and never less than 2.54×. A control that runs e-means' algorithm on o-means' kernels attributes 3.13× to the algorithm and 1.43× to the kernels. At equal time o-means' error is no worse. In a comparison that was not pre-registered, counted in distance computations, which do not depend on hardware, e-means needs at most 2–3 passes to match 25 Lloyd iterations and o-means the equivalent of 0.15–0.45 of a pass.

1 Introduction

Clustering n vectors into k groups with k-means is the usual way to build the coarse quantizer of an inverted-file index (Sivic and Zisserman, 2003; Jégou et al., 2011; Johnson et al., 2021) or a codebook for compression. In these uses k is large: a common rule takes k between $4\sqrt{n}$ and $32\sqrt{n}$, which gives 16,384 centroids for a million vectors and 65,536 for ten million (Bessa et al., 2026). Every method discussed here spends most of its time finding the nearest centroid of each point, which costs k distance evaluations in d dimensions when done exhaustively.

Lloyd's algorithm (Lloyd, 1982) repeats the exhaustive assignment over the full dataset, typically 25 times. e-means (Bessa et al., 2026) reduces the number of passes. It is a mini-batch method in which each centroid is the ratio of two exponential moving averages (EMA), as in the codebook update of VQ-VAE (van den Oord et al., 2017), with a cosine step schedule, revival of dead clusters by splitting, and two fitted power laws that leave the number of epochs T as the only setting. On its authors' four main corpora, from one million to one billion points, it reaches the error of 25 Lloyd iterations within 1.4 to 2.6 epochs (Bessa et al., 2026, Table 4). The cost of one epoch is unchanged: every point of every batch is compared with all k centroids. On the GPU used in this

paper, 80 to 86% of e-means' wall-clock time is spent in the kernel that computes the distances and takes the minimum (one seed, two datasets, $T = 2$; Appendix F).

This paper reduces the cost of an epoch. o-means keeps e-means' objective, starting centroids, update rule, step schedule, batch and step laws and revival rule, and replaces the exhaustive assignment by a two-level one. The k centroids are grouped into $G \approx k/128$ cells by a small k-means run on the centroids themselves. A point is compared with the G cell centres and then only with the centroids of its p nearest cells. For $k = 16{,}384$ and $p = 16$ this is $128 + 16 \times 128 = 2{,}176$ distance evaluations per point instead of 16,384.

Two-level assignment is the search procedure of an inverted-file index, and e-means itself searches an inverted file over the centroids in its multi-GPU configuration, where its authors report a benefit from about two million centroids upward and none at 262,144. The regime studied here, k between 2^{13} and 2^{16} on one GPU, poses a different problem. A fit lasts between about 10 ms and 2 s, so the structure must pay for itself within that time, and it must stay valid while revival relocates most of the centroids it indexes: with $k = 16{,}384$, between 9,000 and 12,000 centroids are split in the first epoch. o-means keeps the centroid table sorted by cell, refreshes the cells after every revival check during the first pass, and records each split as a pointer from the donor centroid to its child. A second, smaller change concerns runs shorter than two epochs, which end with a recency-weighted mean of the whole run.

We compare o-means with e-means under a protocol whose rules and success criteria were locked before any scored run (Section 4). Nine datasets were used for development; eight others were held out and run once, on the final version. All measurements come from a workstation with one NVIDIA RTX 5090 (32 GB). e-means' fastest kernels require an H100, and on the RTX 5090 it runs its Triton backend, so part of any gain measured here is implementation. To separate the two parts we also measure a control that runs e-means' unchanged algorithm on o-means' kernels.

- A two-level assignment for EMA mini-batch k-means at k from 8,192 to 65,536 on one GPU, which performs 6 to 23% of the distance evaluations of an exhaustive pass, with a maintenance schedule and split pointers under which 0.5 to 2.4% of points are assigned to a centroid other than their nearest on four of five development datasets and 4 to 13% on the fifth (Sections 3.1 and 3.2).

- A recency-weighted, split-aware closing mean for runs below two epochs, which lowers the held-out error of one-epoch runs by 0.34 to 0.70% on those five datasets (Section 3.3).

- A comparison with e-means on a locked test suite, with criteria registered beforehand. o-means reaches the error of e-means' 1-, 2- and 3-epoch runs in 1/2.54 to 1/7.25 of e-means' time, and the gain decomposes into 1.43 (kernels) $\times$ 3.13 (algorithm) (Sections 5.1 and 5.3).

- A comparison with Lloyd's algorithm in distance computations and in time, which was not pre-registered (Section 5.2), and negative results on batch size, on exact re-assignment and on the sensitivity of the benchmark to the tested budgets (Section 5.4).

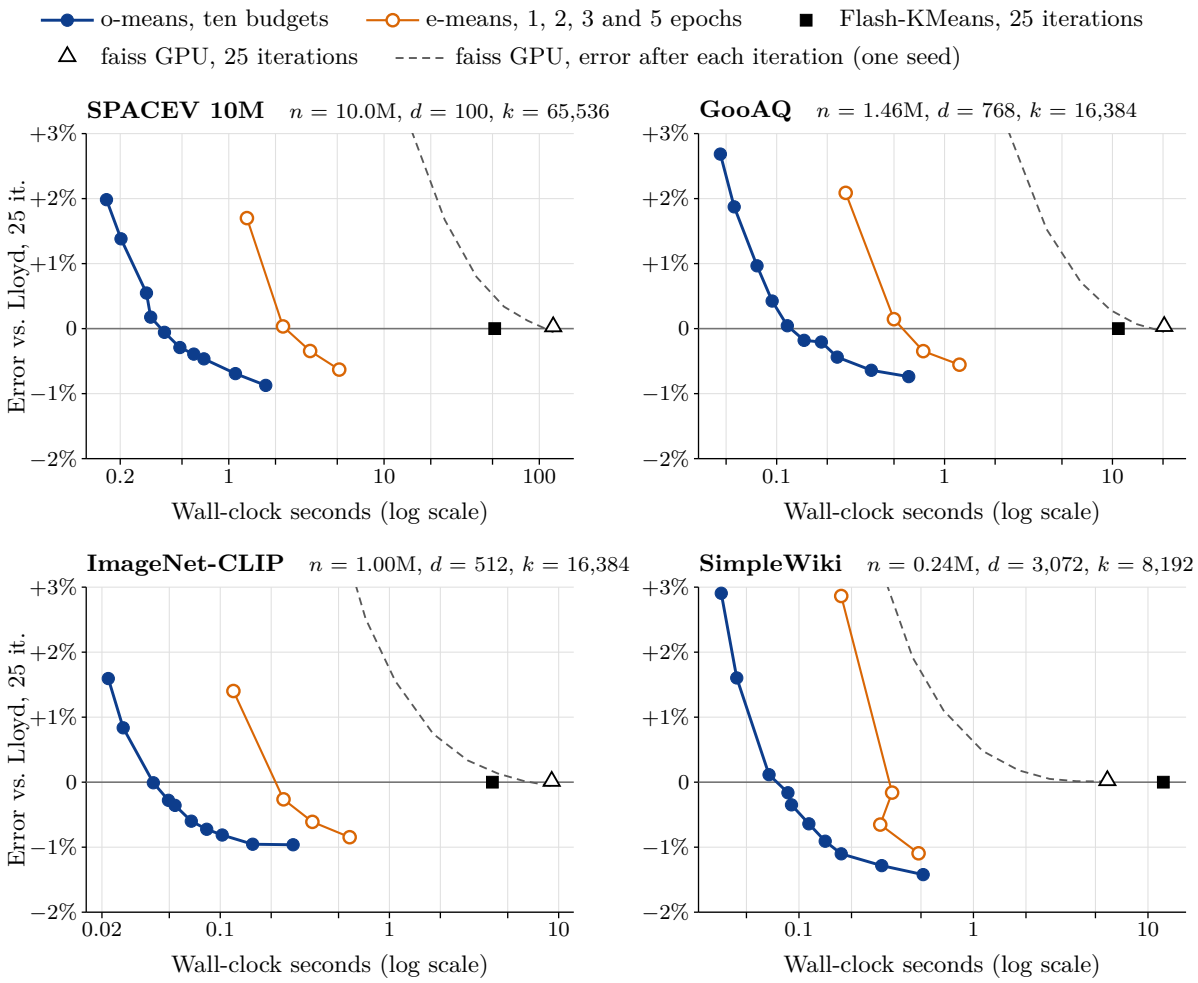


Figure 1. Held-out error against wall-clock time on four of the eight held-out datasets. The vertical axis is the mean squared error relative to 25 Lloyd iterations (Flash-KMeans) from the same start; the horizontal axis is seconds on a logarithmic scale. Lines join the budgets at which e-means ($T = 1, 2, 3, 5$ epochs) and o-means (ten budgets from 0.8 to 12.8 epochs) were run; each point is a mean over seeds. Markers show 25 iterations of Flash-KMeans and of faiss GPU k-means, and the dashed line the error of faiss after each iteration (one seed). One RTX 5090; e-means on its Triton backend.

2 Background

2.1 Problem and notation

Given points $x_1, \dots, x_n$ in d dimensions and a number of clusters k, k-means seeks centroids $C = (c_1, \dots, c_k)$ that minimise the mean squared error

$$\mathrm{MSE}(X, C) = \frac{1}{n} \sum_{i=1}^{n} \|x_i - c_{a(i)}\|^2, \qquad a(i) = \arg\min_j \|x_i - c_j\|^2. \tag{1}$$

We write T for a number of epochs, B for a batch size, G for a number of cells and p for a number of probed cells. A *distance evaluation* is the computation of one $\|x_i - c_j\|^2$, in practice of the score $\|c_j\|^2 - 2x_i^\top c_j$, and costs about $2d$ floating-point operations. A *pass* is one visit of all n points; with exhaustive assignment it costs nk distance evaluations, and we use that quantity as the unit in which distance evaluations are counted.

Lloyd's algorithm (Lloyd, 1982) alternates the assignment $a(\cdot)$ of every point with the replacement of every centroid by the mean of its points, at one exhaustive pass per iteration. The reference quality in the e-means paper and here is 25 Lloyd iterations from k random data points, written Lloyd@25. Mini-batch variants (MacQueen, 1967; Bottou and Bengio, 1995; Sculley, 2010) update the centroids after assigning a batch of $B \ll n$ points; they progress faster than Lloyd at the start but usually stop at a higher error.

2.2 e-means

e-means (Bessa et al., 2026) is the method o-means builds on. We describe it as far as the rest of the paper needs, following the released implementation.

Each cluster j carries a centroid c_j and a mass N_j, an exponential moving average of the number of points it receives per batch. For batch t, let n_j be the number of batch points assigned to j and $\bar{x}_j$ their mean. With step α_t, the update is

$$N_j \leftarrow (1-\alpha_t)\, N_j + \alpha_t\, n_j, \qquad c_j \leftarrow c_j + \frac{\alpha_t\, n_j}{N_j}\,(\bar{x}_j - c_j) \quad \text{if } n_j > 0, \tag{2}$$

where the second expression uses the updated N_j. This is equivalent to keeping exponential moving averages of the per-cluster sum and count and taking their ratio, as in VQ-VAE (van den Oord et al., 2017). The masses start at a small constant, so the first batch moves every centroid it touches onto its batch mean. The step follows a cosine schedule from α_0 to $0.1\,\alpha_0$ over the M updates of the run, $\alpha_t = \alpha_0\big(0.1 + 0.9 \cdot \tfrac{1}{2}(1+\cos(\pi t/M))\big)$.

The batch size and the initial step come from two power laws fitted by the e-means authors,

$$B = \beta\, k^{0.13}\, n^{0.88}\, T^{0.81}, \qquad \alpha_0 = \min\!\big(0.82\, k^{-0.46}\, B^{0.90}\, n^{-0.45}\, T^{-0.11},\ 1\big), \tag{3}$$

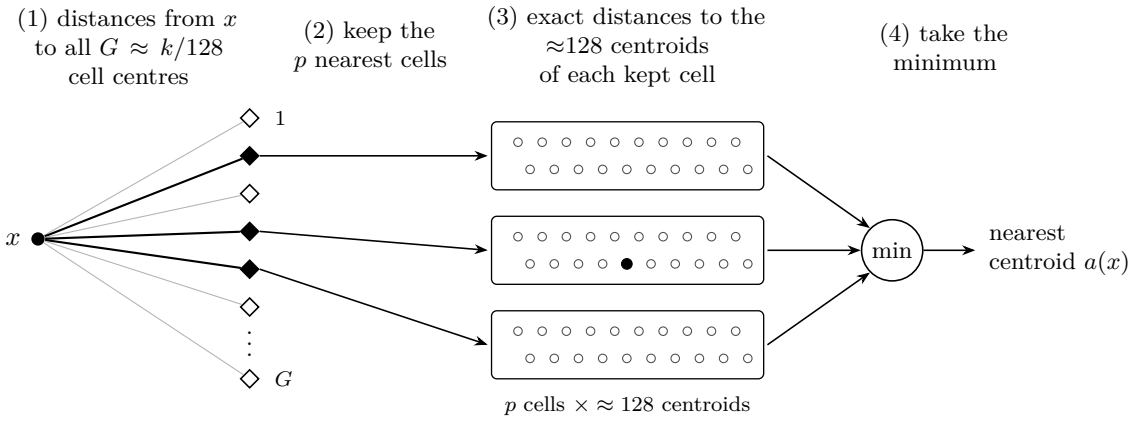
with B rounded to a power of two. The coefficient β is 0.035 in the *throughput* law and one eighth of that in the *quality* law; the e-means benchmarks use the former up to two million points and the latter above. The batches are large and the updates few: a one-epoch run on a million points with $k = 16{,}384$ makes 31 updates with $B = 32{,}768$.

Every five updates, clusters whose mass has fallen below a tenth of the mean mass are declared dead and revived. An equal number of donors is drawn without replacement with probability proportional to mass. Each dead centroid is moved to the position of its donor, the pair is separated by a relative perturbation of 10^{-4} with alternating signs over the coordinates, and the two share the donor's mass equally.

A run of T epochs visits the data in a fresh random order in each epoch; for fractional $T \geq 1$ the fractional pass is placed before a final full pass. At the end, each centroid is replaced by the plain mean of the points assigned to it during the last pass, under the labels those points received when they were processed. We call this the *last-pass mean*. On the GPU the assignment operates on copies of points and centroids in the 16-bit bfloat16 format (BF16). An epoch of e-means costs one exhaustive pass, like a Lloyd iteration.

3 o-means

o-means runs e-means' training loop with unchanged statistics: the starting centroids, the random order of each epoch, the batch size and initial step of Eq. (3), the cosine schedule, the update of Eq. (2) and the revival rule are those of Section 2.2. The one difference in revival is that dead centroids are taken in the order of the centroid table, which o-means keeps sorted by cell, and not



$G + 128\,p$ distances per point instead of k; for $k = 16{,}384$: $G = 128$, $p = 16$, so 2,176 instead of 16,384.

Figure 2. Two-level assignment of one point. The k centroids are grouped into G cells and stored sorted by cell. The point is scored against the G cell centres, keeps its p nearest cells, and is then scored only against the centroids stored in those cells.

in index order; this matters only when dead centroids outnumber donors. o-means changes how a batch is assigned, how runs shorter than two epochs are closed, and how the loop is executed on the GPU. Algorithm 1 gives the complete procedure. The budget of a run is a number of epochs, as in e-means; we write it E and call it the *effort*, to keep it apart from the T of the e-means run it is compared with.

3.1 Two-level assignment

The centroids are partitioned into G cells, each with a cell centre g_h, $h = 1, \dots, G$. The partition is obtained by Lloyd iterations on the k centroids, treated as data, against the G centres. The centroid table and all per-centroid state are kept physically sorted by cell, so that a cell is a contiguous block of rows. A batch point x is assigned in three steps (Figure 2): it is scored against the G cell centres and keeps the p nearest cells; it is scored against the centroids of those p cells and no others; and the best of these is returned, after the additional check of Section 3.2. The numbers of cells and probes are functions of k and d only:

$$G = \min\Big(k,\ \max\Big(16,\ 16\Big\lceil\frac{\lfloor k/128\rfloor}{16}\Big\rceil\Big)\Big), \qquad p = \begin{cases} 24 & \text{if } k > 16{,}384, \\ 12 & \text{if } k \le 16{,}384 \text{ and } d \ge 512, \\ 16 & \text{otherwise,} \end{cases} \tag{4}$$

with p capped at G. The cell count makes the mean cell size 128, which matches the tile sizes of the kernel that scans a cell. Each probe count is the smallest tested value for which fewer than 1% of points missed their nearest centroid in a measurement on fixed centroids of a development dataset, with one seed: 0.4 to 0.5% at $p = 16$ on DEEP 1M and SIFT 1M, 0.5% at $p = 12$ on arXiv-768, and 0.6% at $p = 24$ on DEEP 10M, which is the only dataset behind the rule for $k > 16{,}384$. Inside a fit the rates are higher (Section 3.2).

The cost of an assignment can be estimated directly. If the cells had equal size, one point would cost

$$G + p\,\frac{k}{G} + 2 \;\approx\; \frac{k}{128} + 128\,p \tag{5}$$

distance evaluations in place of k, where the 2 is the split-pointer check. Table 1 evaluates Eq. (5) for the four configurations that occur in our experiments. The implementation counts the evaluations

Table 1. Distance evaluations per point under the cost model of Eq. (5), without the two evaluations of the split-pointer check.

k	d	G	p	$G + p\,k/G$	share of k
8,192	≥ 512	64	12	1,600	19.5%
16,384	< 512	128	16	2,176	13.3%
16,384	≥ 512	128	12	1,664	10.2%
65,536	any	512	24	3,584	5.5%

it performs, including those spent on building the cells and the effect of unequal cell sizes. Over all fits on the 17 datasets of Section 4.2, the measured share of an exhaustive pass is 5.6 to 6.0% for $k = 65{,}536$, 10.5 to 11.7% for $k = 16{,}384$ with $p = 12$, 12.5 to 17.0% with $p = 16$ (13.4 to 14.1% without GloVe-200, which spans the full range), and 20.1 to 23.1% for $k = 8{,}192$.

3.2 Keeping the cells valid

The cells describe where the centroids were when the cells were built. Two things invalidate them. The centroids move after every batch. And when revival splits a donor, the child is written into the table row of the dead centroid it replaces, which belongs to the cell where the dead centroid used to be; a point near the donor does not probe that cell and cannot find the child.

The second effect dominates. In a first version the cells were rebuilt twice per pass. With $k = 16{,}384$, the share of points assigned to a centroid other than their nearest was then 2.3% during the first eighth of the first pass, when the centroids are still close to random data points, and 10 to 28% between one eighth and one half of the first pass, during which 9,000 to 12,000 centroids are split. The error after one epoch rose by 0.53% on DEEP 1M and by 0.76% on arXiv-768 relative to exhaustive assignment.

o-means uses two mechanisms. The first is the rebuild schedule. The first build draws G random centroids as cell centres and runs four Lloyd iterations on the centroids. During the first pass the cells are refreshed after every revival check, that is every five batches, without any Lloyd iteration: each centroid is re-assigned to the nearest existing cell centre and the table is re-sorted, which places every new child in the cell where it now lies. After the first pass the cells are rebuilt every half pass with two Lloyd iterations started from the previous centres.

The second mechanism covers the batches between a split and the next refresh. Each split of a donor v into itself and a child u is recorded as $\text{child}[u] \leftarrow \text{child}[v]$, $\text{child}[v] \leftarrow u$, and the pointers are cleared at every rebuild. After the search of the probed cells, each point is also scored against the child and the grandchild of its best match and keeps the best of the three.

With both mechanisms and the probe counts of Eq. (4), 0.5 to 2.4% of points miss their nearest centroid on DEEP 1M, SIFT 1M, arXiv-768 and Landmark-Nomic-768. On GloVe-200, unit-norm word vectors, the rate is 4 to 13% at $p = 16$. Measured on one seed with exact assignment on training rows, a fit with two-level assignment ends 0.02 to 0.13% higher in training error than the same fit with exhaustive assignment after the same number of epochs, and takes between a quarter and a half of the time (Appendix F).

3.3 Closing mean for short runs

e-means' last-pass mean averages each point under the label it received when it was processed. In a run of one epoch the last pass is the whole run. It then includes the points labelled against the starting centroids, and the points that a centroid won before revival moved it elsewhere.

Algorithm 1 o-means. Lines 1, 6 and 7 and the choice of donors in line 9 are e-means' (Bessa et al., 2026); $\bar{x}_j$ is the mean of the batch points labelled j. A rebuild is due at $t = 1$, every five batches during the first pass, and every half pass afterwards; r_t is the number of rows processed through batch t and R the total number of rows of the run.

Require: data X ($n \times d$, on the GPU), k, starting centroids C, effort E, seed
1: plan the epochs, batches $\mathcal{B}_1, \dots, \mathcal{B}_M$ and steps $\alpha_1, \dots, \alpha_M$ of an E-epoch e-means run (Section 2.2)
2: $G, p \leftarrow$ Eq. (4); $N_j \leftarrow$ small constant; closing sums $A_j \leftarrow 0$ and masses $w_j \leftarrow 0$
3: **for** $t = 1, \dots, M$ **do**
4: **if** a rebuild is due **then** BuildCells(I) with $I = 4$ at $t = 1$, $I = 0$ during the first pass, $I = 2$ after it
5: $\ell(x) \leftarrow$ Assign(x) for every $x \in \mathcal{B}_t$
6: $n_j \leftarrow |\{x \in \mathcal{B}_t : \ell(x) = j\}|$, $\bar{x}_j \leftarrow \frac{1}{n_j} \sum_{x \in \mathcal{B}_t : \ell(x) = j} x$
7: update N_j and c_j by Eq. (2) with step α_t
8: $A_j \leftarrow A_j + \omega_t n_j \bar{x}_j$, $w_j \leftarrow w_j + \omega_t n_j$, with $\omega_t = e^{-(R - r_t)/n}$ if $E < 2$; otherwise $\omega_t = 1$ in the last pass and 0 before it
9: **if** $t \bmod 5 = 0$ **then** for each dead centroid u and its donor v: split v into v and u; child[u] $\leftarrow$ child[v]; child[v] $\leftarrow u$; if $E < 2$: $A_v, A_u \leftarrow A_v/2$ and $w_v, w_u \leftarrow w_v/2$
10: **return** $c_j \leftarrow A_j / w_j$ for every j with $w_j > 0$; the other centroids keep their position
11: **function** BuildCells(I)
12: cell centres $g \leftarrow$ the previous centres, or G random centroids at the first build
13: repeat I times: assign each centroid to its nearest centre; move each non-empty centre to the mean of its centroids
14: assign each centroid to its nearest centre; sort the centroid table and all per-centroid state by cell
15: child[j] $\leftarrow \varnothing$ for all j
16: **function** Assign(x)
17: $P \leftarrow$ the p cells h with the smallest $\|g_h\|^2 - 2x^\top g_h$
18: $j_0 \leftarrow \arg\min_j \|c_j\|^2 - 2x^\top c_j$ over the centroids j stored in the cells of P
19: $j_1 \leftarrow$ child[j_0], $j_2 \leftarrow$ child[j_1] ▷ where defined
20: **return** the best of j_0, j_1, j_2 under the same score

Below two epochs o-means closes differently. Every batch of the run contributes to the closing mean with weight

$$\omega_t = \exp\left(-\frac{R - r_t}{n}\right), \tag{6}$$

where r_t is the number of rows processed through the end of batch t and R the total number of rows of the run. The time constant is one pass. When a centroid is split, donor and child each keep half of the donor's accumulated closing sum and closing mass.

On the five development datasets on which it was designed, with exhaustive assignment, this mean lowers the validation error of a one-epoch run by 0.34 to 0.70% relative to the last-pass mean. On the test suite a one-epoch o-means run ends 0.07 to 1.2% below e-means' one-epoch error, and a 0.8-epoch run 0.04 to 0.80% above it. The benefit does not extend to longer runs. From 1.6 epochs upward its effect on the validation error was between −0.18 and +0.03%, and at two epochs it raises the training error by 0.24 to 0.32% (two datasets), because it retains points labelled more than a pass earlier. From two epochs upward o-means therefore uses e-means' last-pass mean.

3.4 Kernels and numerics

The implementation is written in PyTorch (Paszke et al., 2019) with kernels in Triton (Tillet et al., 2019). The data are converted once to a half-precision (FP16) copy on the GPU, and every kernel reads its rows through the permutation of the current epoch, so no batch is gathered or converted during training. Points and centroids are compared in FP16 with accumulation in single precision (FP32), and batch sums are accumulated exactly in 64-bit fixed point. The two-level assignment is three kernel launches per batch: one scores the cell centres and groups the (point, cell) pairs by cell, one scans each probed cell's block of centroids on tensor cores, and one reduces the partial results and performs the split-pointer check. No value is read back to the host inside the loop. Appendix F gives the details.

To measure what the kernels contribute without the algorithmic changes, we define a control called *v0*: e-means' algorithm, with exhaustive assignment and the last-pass mean, on o-means' kernels. Its assignment kernel runs at 193 to 224 TFLOPS (10^{12} floating-point operations per second, at $2d$ per distance evaluation) on the RTX 5090, where the half-precision matrix product of cuBLAS, NVIDIA's dense linear-algebra library, reaches about 230 and e-means' Triton kernel reaches 104 to 170 inside a fit. v0 differs from e-means in precision (FP16 instead of BF16, exact sums) but not in its statistics: on the development datasets its mean error after T epochs is within $\pm 0.05\%$ of e-means', with one exception at -0.13%.

3.5 Budgets

The effort E may be fractional; below one epoch the batch size and initial step of a one-epoch run are used, as in the e-means benchmark code. A time-to-error comparison needs o-means' error at several budgets, and we run it at the ten efforts

$$E \in \{0.8,\ 1,\ 1.6,\ 2,\ 2.5,\ 3.2,\ 4,\ 5,\ 8,\ 12.8\}, \tag{7}$$

which we call the *effort ladder*. It was chosen on the development datasets so that neighbouring points are at most about 1.5 times apart in time up to five epochs; the points beyond five epochs exist because e-means' error is nearly flat between five and eight epochs. The ladder belongs to the benchmark and not to the algorithm (Section 5.4).

4 Evaluation methodology

The protocol follows the published benchmark rules of the e-means paper wherever they exist. It was locked, with its success criteria, before any scored run (Section 4.5). This section states the rules that determine what a reported number means. Appendix A gives the remaining details and lists the deviations from the e-means paper.

4.1 Task, hardware and reference

An algorithm receives the data as an FP32 tensor resident on the GPU, k, k starting centroids, a budget and a seed, and returns k centroids. All runs use a workstation with one NVIDIA RTX 5090 (32 GB) and 32 CPU threads. The reference is the released e-means implementation, called as in its own benchmark: BF16 assignment, the throughput batch law for $n \leq 2$ million and the quality law above, and the fastest backend the GPU supports. On an RTX 5090 that backend is Triton; e-means' fastest kernels, written with the CuTe layer of NVIDIA's CUTLASS library, run only on H100-class GPUs. All speed-ups in this paper are therefore specific to this machine.

Table 2. The 17 datasets. The nine development datasets were used for all design decisions and tuning; the eight test datasets were held out until the method was frozen. Seeds are the number of starting configurations per dataset.

Dataset	Suite	n	d	k	Seeds	Description
SPACEV 1M	test	1.00M	100	16,384	5	SPACEV web search, 1M subset
SPACEV 10M	test	10.0M	100	65,536	3	SPACEV web search, 10M subset
ImageNet-CLIP	test	1.00M	512	16,384	5	ImageNet images, CLIP embeddings
GooAQ	test	1.46M	768	16,384	5	VIBE GooAQ, DistilRoBERTa
CodeSearchNet	test	1.35M	768	16,384	5	VIBE CodeSearchNet, Jina, cosine
AG-News	test	0.75M	1,024	16,384	5	VIBE AG News, mxbai
Landmark-DINO	test	0.74M	768	16,384	5	VIBE Landmarks, DINO, cosine
SimpleWiki	test	0.24M	3,072	8,192	5	VIBE Simple Wikipedia, OpenAI
DEEP 1M	dev.*	1.00M	96	16,384	5	DEEP1B image descriptors
SIFT 1M	dev.*	1.00M	128	16,384	5	BigANN SIFT, scaled by 1/512
GloVe-200	dev.*	1.17M	200	16,384	5	VIBE GloVe word vectors, cosine
arXiv-768	dev.*	1.32M	768	16,384	5	VIBE arXiv abstracts, Nomic
Landmark-Nomic	dev.*	0.74M	768	16,384	5	VIBE Landmarks, Nomic
DEEP 10M	dev.	10.0M	96	65,536	3	DEEP1B image descriptors
SIFT 10M	dev.	10.0M	128	65,536	3	BigANN SIFT, scaled by 1/512
CC-News	dev.	0.48M	768	8,192	5	VIBE CC-News, Nomic
CelebA	dev.	0.18M	2,048	8,192	5	VIBE CelebA faces, ResNet, cosine

* Also in the five-dataset fast development suite (3 seeds each).

4.2 Datasets and the development/test split

Table 2 lists the 17 datasets. They are taken from the experiments of the e-means paper: DEEP (Babenko and Lempitsky, 2016), SIFT and SPACEV as distributed for the billion-scale search benchmarks (Simhadri et al., 2022), and ImageNet-CLIP and ten further embedding sets from the VIBE benchmark (Jääsaari et al., 2025). As in the e-means paper, $k = 16{,}384$ for $n \leq 2$ million and 65,536 for 10 million points, except that the three smallest VIBE sets use $k = 8{,}192$.

Nine datasets form the development suite, *dev-full*. Five of them, *dev-fast*, were run in every development round with three seeds; all nine were used to confirm a round before release. The other eight datasets form the test suite. The assignment of datasets to suites was fixed before the lock. Every dataset has a set of evaluation rows that are never trained on, split once into a validation half and a test half with a fixed permutation. Development datasets are scored on their validation half and test datasets on their test half. Everything that was fitted or chosen, including Eq. (4), the rebuild schedule, the two-epoch threshold of the closing mean and the effort ladder, was fitted on the development suite.

4.3 Measurement

Both methods start from the same k centroids, the first k rows of a seeded permutation of the training rows as in the e-means benchmark. One process handles one (dataset, seed) pair. The data are loaded and moved to the GPU before any timing. In that process e-means is run at $T \in \{1, 2, 3, 5\}$ and o-means at the ten efforts of Eq. (7), interleaved so that drift in the machine affects both.

The timed interval of a fit covers everything the method does: conversion of the data, construction of its state, all passes and the closing step. It excludes loading the data, drawing the starting centroids and scoring. Each fit is preceded by an untimed identical warm-up call, and a fit that

takes less than 2 s is timed three times and the median is kept.

The error of a set of centroids is the MSE of Eq. (1) on the evaluation rows with *exact* assignment: centroids are ranked in IEEE single precision and the winning distance is recomputed in double precision, as in the e-means benchmark. The approximate assignment of o-means is never used for scoring. We call this the *held-out error*.

4.4 Time to e-means' error and error at equal time

For each dataset the seeds are pooled first. Let $\bar{t}_e(T)$ and $\bar{m}_e(T)$ be the mean time and mean held-out error of e-means at T epochs, and let (t_E, m_E) be the mean time and mean error of o-means at effort E. The *envelope* $\mathrm{env}(t)$ is the best error among the o-means points with $t_E \le t$. Between two adjacent envelope points whose times differ by a factor of at most 1.6, env is interpolated linearly in $(\log t, \text{error})$. Across a wider gap it is a step, so no credit is given before the later point is reached. This is the *interpolation limit*.

The *crossing time* $t^*(T)$ is the earliest time at which $\mathrm{env}(t) \le \bar{m}_e(T)$, and never earlier than the cheapest ladder point. The *speed-up at* T is

$$s(T) = \frac{\bar{t}_e(T)}{t^*(T)}, \tag{8}$$

the ratio of e-means' time for T epochs to the time o-means needs to reach the same held-out error. A target that is never reached scores at most 0.5. The summary statistic, which the protocol calls the *score*, is the geometric mean of $s(T)$ over all datasets of a suite and $T \in \{1, 2, 3\}$. In the rest of the paper we call it the geometric-mean speed-up. A *target* is one (dataset, T) pair. The *strict* speed-up uses the step envelope everywhere, so only budgets that were actually run count. *Per-seed* speed-ups are computed for each seed against its own paired e-means run, and a seed counts as a win if its speed-up exceeds 1.

The *error at equal time* is the relative difference

$$\mathrm{gap}(T) = \frac{\mathrm{env}(\bar{t}_e(T)) - \bar{m}_e(T)}{\bar{m}_e(T)}, \tag{9}$$

which is negative when o-means has the lower error at the moment e-means finishes T epochs.

The targets are $T = 1, 2, 3$ and not $T = 5$, for a reason measured before the lock: a copy of e-means that differs only in its internal seed never reached the paired e-means' five-epoch error within 6.25 epochs on 5 of 15 (dataset, seed) pairs, so time to that error is unstable even for an identical algorithm (Appendix E). $T = 5$ is used for the error at equal time, gap(5), and the speed-up at $T = 5$ is reported without entering the geometric mean. As a hardware-independent cost we also report the number of distance evaluations of the o-means run at which the target is reached, counted at the first ladder point at or beyond the crossing, as a fraction of e-means' nkT.

4.5 Noise floor, criteria and the test run

The noise of the harness was measured before the lock by running e-means as its own candidate (Appendix E). An exact copy of e-means has a geometric-mean speed-up of 1.0001. A copy with a different internal seed, which is statistically equivalent but independent, defines two tolerances: $\varepsilon = 0.013$ for the geometric-mean speed-up and $\varepsilon_{\mathrm{gap}} = 0.16\%$ for gap(5). The same control shows that the crossing rule is conservative: an independent copy of e-means measures 0.95 to 0.99 and not 1.0, because interpolation along a chord of a convex curve overestimates the error between ladder points.

Table 3. The three pre-registered criteria and the result of the single release run on the held-out test suite.

Pre-registered criterion	Bar	Held-out test result
Speed-up in time to e-means' error, geometric mean over 8 datasets and $T = 1, 2, 3$	≥ 1.15	4.49
Smallest single speed-up (one dataset, one T)	≥ 0.95	2.54 (SimpleWiki, $T = 3$)
Largest error difference at e-means' 5-epoch time	$\leq +0.16\%$	-0.12%

Three success criteria were registered: a geometric-mean speed-up of at least 1.15 on the test suite; no target below 0.95; and gap(5) no larger than $+\varepsilon_{\text{gap}}$ on any test dataset. During development, a change was kept if it improved the geometric-mean speed-up on dev-fast by more than 3% without any guardrail regressing beyond noise, and a kept change was confirmed on dev-full before release.

The protocol was locked on 12 November 2026 at 13:53 UTC, and the first scored round ran at 14:01. The locked text includes amendments made after the noise controls: the pooling of seeds and the restriction of the targets to $T \leq 3$. Unscored timings of the kernel control against e-means existed before the lock: a prototype on synthetic data the day before, and v0 through the harness on dev-fast. Both gave about 1.5. The registered bar of 1.15 was therefore below what the kernels alone were known to give, and meeting it is weak evidence by itself; the informative results are the measured ratios and their decomposition. A recorded update at 15:50 completed the reference files without changing scoring code or thresholds (Appendix A).

The test suite was run once for the release, on the final code, on the same day. No test data were used for tuning, and the release run was not repeated. The control v0 was run on the test suite afterwards, for the decomposition of Section 5.3 only. Every run on the test suite is recorded in an append-only ledger. The comparison with Lloyd's algorithm in Section 5.2 was not part of the pre-registered protocol.

5 Results

5.1 Against e-means

All three criteria are met (Table 3); as Section 4.5 explains, the first was already within reach of the kernels alone. All 38 runs of the release completed, no ladder was truncated, no output contained a non-finite value, and the peak GPU memory was 7.5 GB including the data.

Figure 3 shows, for each test dataset and target, the time e-means takes and the time at which o-means reaches the same held-out error; Table 4 gives the ratios. On GooAQ (1.46 million vectors, $d = 768$, $k = 16{,}384$) e-means takes 0.50 s for two epochs and o-means reaches that error in 0.11 s. On SPACEV 10M ($k = 65{,}536$) the times are 2.23 s and 0.36 s. Figure 1 shows the full curves for four datasets. Over the 24 targets the speed-up ranges from 2.54 to 7.25 with geometric mean 4.49; with the strict rule the geometric mean is 4.08. By target it is 4.89 at one epoch, 4.53 at two and 4.09 at three. The seven datasets with at most 1.5 million vectors have a geometric mean of 4.25, and SPACEV 10M, the only test dataset with 65,536 centroids, has 6.55. Each seed of each dataset is faster than its own paired e-means run at all three targets, 114 comparisons out of 114; the smallest per-seed speed-up is 2.32.

Differences below about 0.1 are timing noise: the same code, run twice on dev-fast with bitwise-identical centroids, measured 4.54 and 4.44. The suites that were used for tuning give 4.44 (dev-fast) and 4.61 (dev-full), and the held-out value lies between them. Figure 4 shows the speed-ups for all 17 datasets and Appendix B the development table. The smallest speed-up for any target on the 17 datasets is 2.46.

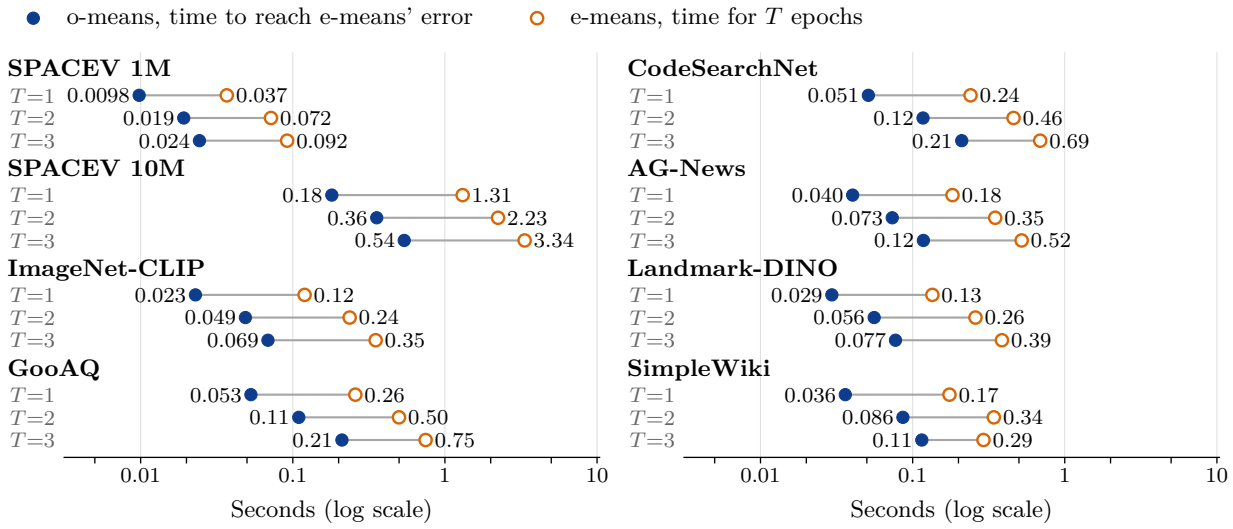


Figure 3. Time to the same held-out error on the eight test datasets. For each dataset and each target $T = 1, 2, 3$, one marker shows the time e-means takes for T epochs and the paired marker the time at which o-means reaches the same error (crossing time, Section 4.4). Seconds, logarithmic axis; means over seeds.

Table 4. Held-out test suite, release run. Speed-up is e-means' time for T epochs divided by the time o-means needs to reach the same held-out error (Eq. 8); $T = 5$ is reported but does not enter the geometric mean. The error column is gap(5), o-means' error relative to e-means' at e-means' 5-epoch time (Eq. 9). The distance ratio is the number of distance evaluations o-means used to reach the target, as a fraction of e-means'.

			Speed-up at T epochs				Error at T=5	Distance ratio at T epochs		
Dataset	n	k	1	2	3	5†	(%)	1	2	3
SPACEV 1M	1.00M	16,384	3.77	3.74	3.77	3.54	−0.13	0.14	0.17	0.15
SPACEV 10M	10.0M	65,536	7.25	6.28	6.18	5.30	−0.24	0.06	0.07	0.08
ImageNet-CLIP	1.00M	16,384	5.22	4.84	5.11	5.14	−0.12	0.11	0.11	0.15
GooAQ	1.46M	16,384	4.86	4.56	3.55	4.09	−0.19	0.11	0.13	0.19
CodeSearchNet	1.35M	16,384	4.69	3.94	3.28	3.73	−0.16	0.11	0.18	0.19
AG-News	0.75M	16,384	4.53	4.75	4.42	5.27	−0.49	0.11	0.11	0.15
Landmark-DINO	0.74M	16,384	4.58	4.63	5.00	4.37	−0.18	0.11	0.13	0.14
SimpleWiki	0.24M	8,192	4.84	3.96	2.54	2.80	−0.19	0.22	0.22	0.30
Geometric mean			4.89	4.53	4.09	4.19		0.11	0.13	0.16

† Not part of the pre-registered criteria (unscored).

The gain is smallest where k is small. Three of the 17 datasets have $k = 8{,}192$: SimpleWiki in the test suite, CC-News and CelebA in development. Their per-dataset geometric means, 3.09 to 3.65, are the three lowest, against 3.76 to 7.83 for the other fourteen, and SimpleWiki at $T = 3$ is the weakest test target at 2.54. This follows from the cost model: with 64 cells and 12 probes a point is still compared with about a fifth of the centroids (Table 1).

Figure 5 shows o-means' held-out error relative to e-means' at the moments e-means finishes 1, 2, 3 and 5 epochs. On the test suite it is 2.0 to 3.9% lower at e-means' one-epoch time, 0.69 to 1.3% lower at two epochs, 0.35 to 0.80% lower at three, and 0.12 to 0.49% lower at five. The differences shrink with the budget because both error curves flatten. The noise floor of this comparison at five

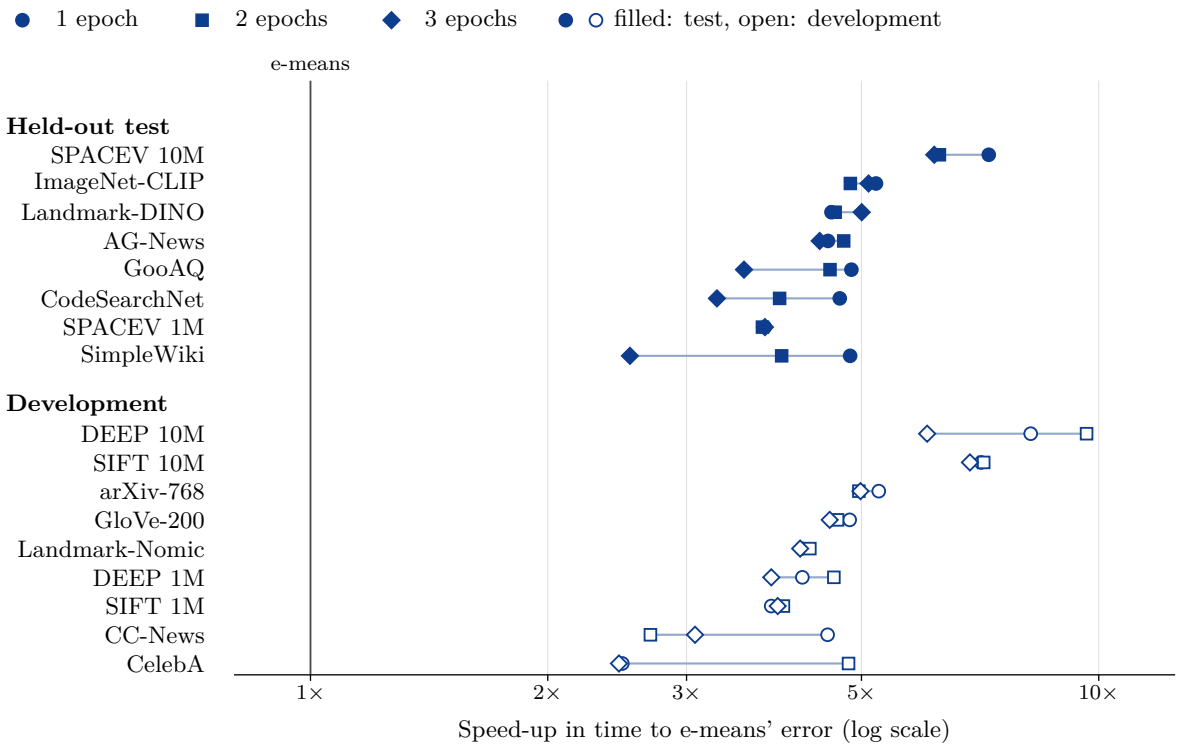


Figure 4. Speed-up of o-means over e-means in time to e-means' held-out error at $T = 1, 2, 3$ epochs, for the eight test datasets and the nine development datasets. The three datasets with $k = 8{,}192$ have the lowest per-dataset means and the three with $k = 65{,}536$ the highest.

epochs is 0.16%. Two of the eight margins at five epochs, SPACEV 1M and ImageNet-CLIP, are inside it, as are six of the 17 when the development datasets are included. For long budgets the supported statement is therefore that o-means is not worse at equal time, not that it is better. On the fixed training subset the corresponding difference at five epochs is −0.31% on average.

e-means' own run time is not monotone in T on small datasets. On SimpleWiki three epochs run faster than two (0.29 s against 0.34 s) because the batch law doubles the batch size, which favours o-means at the two-epoch target. If each target's e-means time is replaced by the shortest e-means time, over $T \in \{1, 2, 3, 5\}$, that reaches the target's error, the geometric mean on the test suite is 4.46. The ladder, moreover, stops at 12.8 epochs. On four of the eight test datasets e-means at eight epochs ends 0.02 to 0.16% below every point of o-means' ladder, at 2.7 to 3.2 times the time of o-means' longest run. We have not shown that o-means reaches that error at any budget.

5.2 Against Lloyd's algorithm

The e-means paper's reference for Lloyd's algorithm on a GPU is Flash-KMeans (Yang et al., 2026), version 0.3.1 with half-precision data, run for 25 full-batch iterations from the same starting centroids. That run was recorded for every dataset and seed when the benchmark was built, as an anchor. Here we ask how much work and time e-means and o-means need to reach its held-out error. This comparison was not among the pre-registered criteria; it was computed afterwards from runs that already existed, plus the faiss measurements described below.

We count distance computations first, because the count does not depend on any implementation. Lloyd@25 costs 25 passes. On the eight test datasets, the first measured e-means budget at or below Lloyd@25's error is 2 or 3 epochs, that is 2 or 3 passes. These are upper bounds: e-means

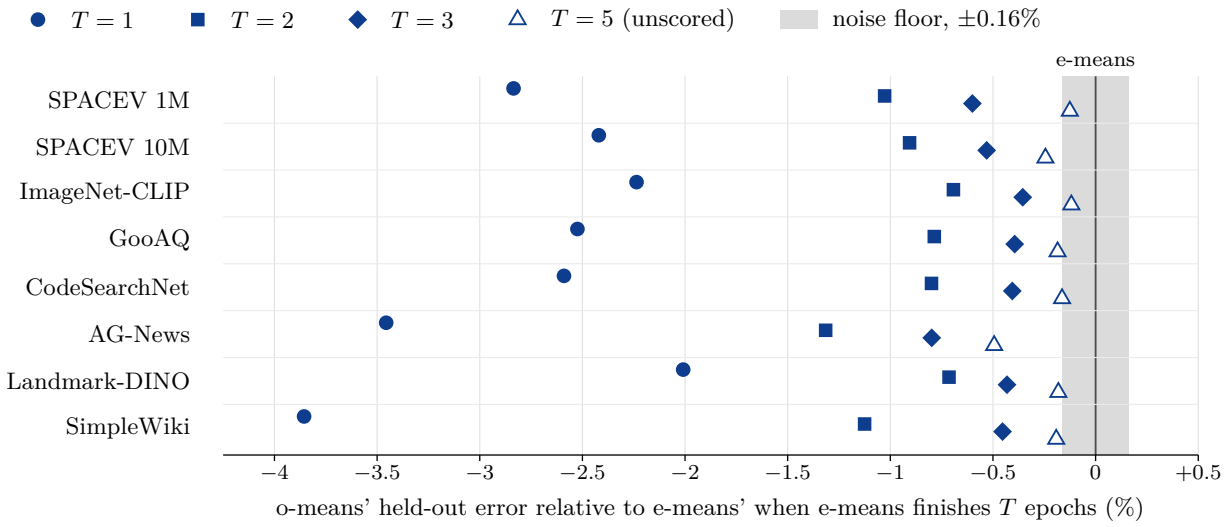


Figure 5. Held-out error of o-means relative to e-means at equal wall-clock time, on the eight test datasets: the value of Eq. (9) at e-means' 1-, 2-, 3- and 5-epoch times. Negative values mean o-means has the lower error. The band marks the ±0.16% noise floor measured for this comparison at five epochs.

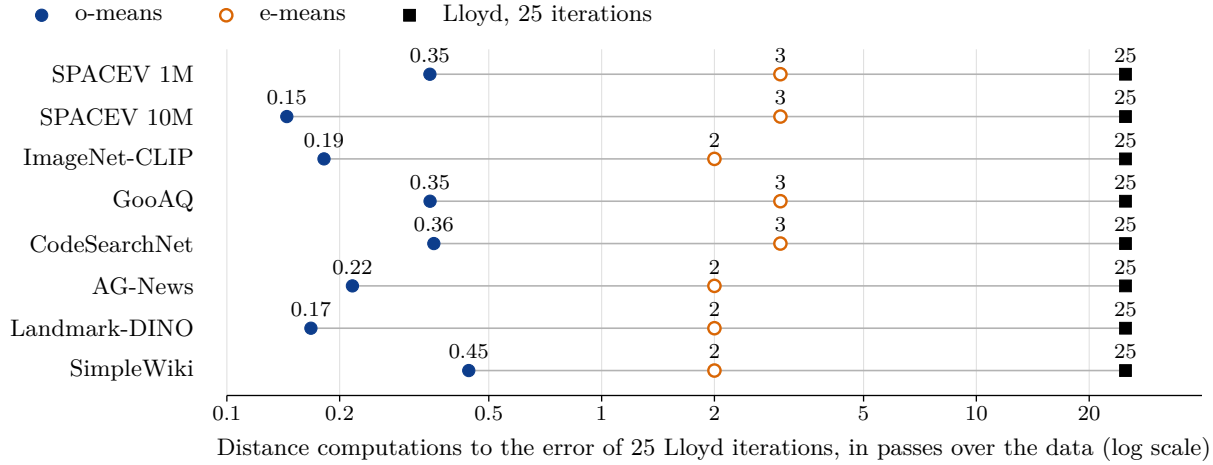


Figure 6. Distance computations needed to reach the held-out error of 25 Lloyd iterations, in units of one exhaustive pass (nk evaluations), for Lloyd, e-means and o-means on the eight test datasets. Logarithmic axis. The counts do not depend on hardware.

was measured only at $T \in \{1, 1.5, 2, 3, 5, 8\}$, while o-means was measured on a finer ladder, and the e-means paper reports fractional values. o-means reaches the same error at an effort between 1.6 and 3.2 epochs, each of which costs a fraction of an exhaustive pass, for a total equivalent to 0.15 to 0.45 passes (Figure 6).

Table 5 and Figure 7 give the times. On the RTX 5090 Flash-KMeans takes 1.35 to 51.6 s for its 25 iterations, and o-means reaches the same error in 0.020 to 0.37 s, which is 69 to 274 times sooner (geometric mean 126). By the same measure e-means is 27 times sooner than Flash-KMeans in the geometric mean. These multiples overstate what the algorithms alone account for. Flash-KMeans' half-precision distance kernel runs at about 25 to 103 TFLOPS on this GPU, where the half-precision matrix product of cuBLAS reaches about 230, and its 25 iterations take roughly 4 to 16 times longer here than the e-means paper reports for the same datasets on an H100. Part of each multiple

Table 5. Lloyd's algorithm on the test datasets (not pre-registered). Time of 25 iterations of Flash-KMeans and of faiss GPU k-means from the same start, time for e-means and o-means to reach the held-out error of the Flash-KMeans run, and the distance computations each needs, in passes. Flash-KMeans runs in half precision at well below the half-precision capability of the RTX 5090, and faiss was run with its default single-precision search (see text).

	Lloyd, 25 iterations (s)		Same error (s)		Distance computations (passes)		
Dataset	Flash-KMeans	faiss GPU	e-means	o-means	Lloyd	e-means	o-means
SPACEV 1M	1.35	3.09	0.073	0.020	25	3	0.35
SPACEV 10M	51.61	122.99	2.331	0.367	25	3	0.15
ImageNet-CLIP	4.06	9.10	0.195	0.041	25	2	0.19
GooAQ	10.84	20.33	0.564	0.122	25	3	0.35
CodeSearchNet	11.05	18.92	0.476	0.120	25	3	0.36
AG-News	18.67	12.43	0.318	0.069	25	2	0.22
Landmark-DINO	6.95	10.29	0.206	0.041	25	2	0.17
SimpleWiki	12.25	5.85	0.272	0.075	25	2	0.45

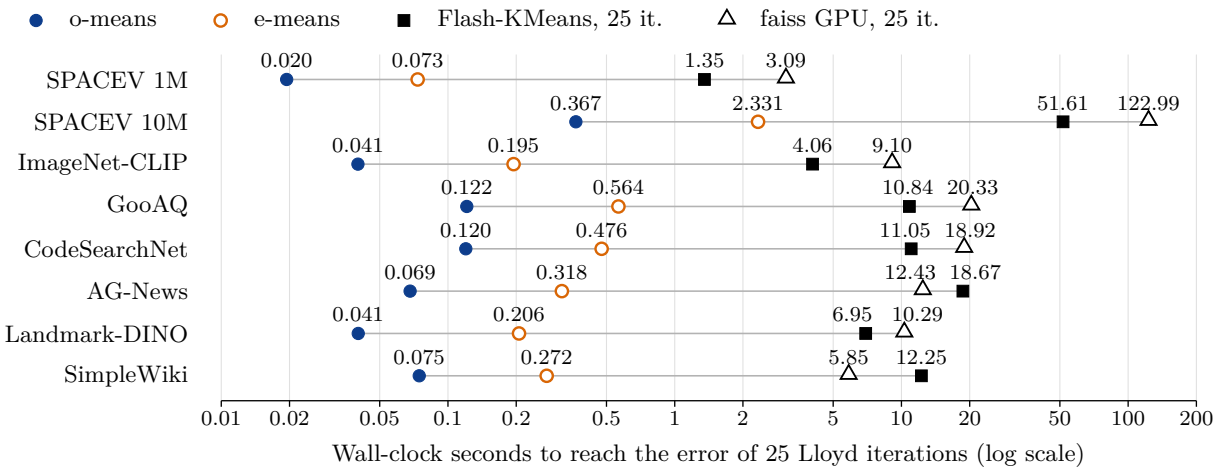


Figure 7. Wall-clock seconds to reach the held-out error of 25 Lloyd iterations on the eight test datasets: Flash-KMeans and faiss GPU (time of their 25 iterations), e-means and o-means (crossing times). Logarithmic axis; one RTX 5090. Flash-KMeans runs at about 25 to 103 TFLOPS in half precision and faiss at 26 to 52 in single precision on this GPU, so the distances between the Lloyd markers and the others include implementation effects.

reflects slow baselines on this card.

Because faiss (Johnson et al., 2021; Douze et al., 2025) is the most widely used GPU k-means, we also timed it: faiss 1.14.3 from conda-forge, its own k-means loop with its default brute-force single-precision search on the GPU, 25 iterations from the same start with the data already on the device, timed as the other methods. Its half-precision option was not tried. Its error is within 0.02% of Flash-KMeans', so the two agree on what 25 iterations achieve. faiss takes 3.09 to 123 s. It is slower than Flash-KMeans on six of the eight test datasets and faster on the two with the highest dimension, AG-News and SimpleWiki. o-means reaches its error 78 to 337 times sooner (geometric mean 181). The caution about slow baselines applies here too: faiss's search runs at 26 to 52 TFLOPS, a single-precision rate that should not be set against the half-precision figure above.

At the end of its ladder, 12.8 epochs, o-means is 0.73 to 1.65% below Lloyd@25's error. It gets

Table 6. Decomposition of the geometric-mean speed-up over e-means into kernels (the control v0, e-means' algorithm on o-means' kernels, against e-means) and algorithm (o-means against v0), on the three suites.

Suite	Datasets	Runs	Speed-up in time to e-means' error (geometric mean)		
			v0 vs. e-means (kernels)	o-means vs. e-means	o-means vs. v0 (algorithm)
Development, fast	5	15	1.51	4.44	2.94
Development, full	9	41	1.53	4.61	3.02
Held-out test	8	38	1.43	4.49	3.13

there in 15 to 44 times less time than the 25 iterations of Flash-KMeans, a multiple that carries the same caveat about the baseline.

5.3 Kernels and algorithm

The reference e-means cannot use its H100 kernels on the RTX 5090, so the speed-up of Section 5.1 mixes a better implementation for this card with a cheaper algorithm. The control v0 (Section 3.4) runs e-means' algorithm on o-means' kernels. At the points where it matches e-means' error on the test suite it has performed between 1.0 and 1.33 times e-means' distance evaluations, never fewer, so whatever it gains over e-means is due to the kernels.

On the test suite v0 has a geometric-mean speed-up of 1.43 over e-means, and o-means is 3.13 times faster than v0 (Table 6). The split varies with k: per dataset, the algorithmic factor is 1.96 on SimpleWiki ($k = 8{,}192$) and 4.89 on SPACEV 10M ($k = 65{,}536$).

These factors need not transfer to other hardware. On an H100, where e-means uses its CuTe kernels, the kernel factor would probably be smaller; we could not measure it. The algorithmic factor has support that does not depend on the card: at the crossings o-means has performed 0.06 to 0.30 of e-means' distance evaluations on the test suite, 0.13 in the geometric mean. But about 7 times fewer distance evaluations bought a threefold reduction in time, because the two-level kernels read rows scattered through the data, work on cells of unequal size, and pay for the rebuilds, which take a quarter of a one-epoch fit on DEEP 1M (Appendix F). How this exchange rate carries over to other GPUs is not measured.

5.4 Ablations and discarded variants

Each development round changed one thing relative to the best kept state and was measured on dev-fast by the full protocol; Appendix C lists the rounds. The kernels alone (v0) give a geometric-mean speed-up of 1.51 over e-means. Adding the closing mean of Section 3.3 to v0 gives 1.66, through the one-epoch targets, which improve by 13 to 20%. Replacing the exhaustive assignment by the two-level assignment, with e-means' last-pass mean, gives 4.29. Adding the closing mean below two epochs gives 4.54, again through the one-epoch targets.

Three variants were measured and not adopted. Smaller batches were considered because o-means' updates are cheap. A simulation on synthetic data predicted that quartering the batch would lower the error at two epochs by 0.56%. On the five dev-fast datasets, with the batch divided by four at $d \leq 256$ and by two at $d = 768$, the measured change was between -0.04 and $+0.16$%, for 3 to 18% more time. The idea was abandoned by a criterion written down beforehand, without a scored round. Exact re-assignment of the oldest quarter of the last pass against the final centroids lowers the error by 0.23 to 0.43% at 1.25 epochs, but adds 19 to 20% to the time of the fit; the geometric-mean speed-up fell by 3.8% and the change was reverted. An adaptive variant that

measures the miss rate during the fit and raises p recovers 0.04 to 0.06% of error on GloVe-200 for 55 to 64% more time, and costs 2 to 4% on the datasets where it leaves p unchanged; it is not the default.

The benchmark is sensitive to where the tested budgets lie. The first version of the closing mean was run on a ladder with a factor-of-two hole between 3.2 and 6.4 epochs. It improved the geometric mean by 6%, but one dataset's three-epoch target fell into the hole, the interpolation limit applied, its speed-up dropped to 0.69, and the round failed its guardrail. The same algorithm on a ladder without the hole improved the geometric mean by 10.2%. Ladder placement is thus worth about 4% of the geometric mean.

6 Limitations

- All timings come from one RTX 5090, with e-means on its Triton backend. We have no H100 measurements; the kernel factor (1.4 to 1.5) would probably shrink there, and the exchange rate between distance evaluations and time is unmeasured on other hardware.
- The assignment during training is approximate. Under exact scoring, o-means' error at e-means' five-epoch time was at or below e-means' on all 17 datasets, but there is no worst-case guarantee. The labels returned by training come from the approximate search.
- The probe rule of Eq. (4) rests on single-seed measurements on a few development datasets, and for $k > 16{,}384$ on DEEP 10M alone. On GloVe-200, 4 to 13% of points miss their nearest centroid, at a cost of about 0.1% in error, and a synthetic mixture with more modes than clusters loses more (Appendix D).
- On synthetic, well-separated mixtures in 1,024 dimensions and more, a few cells collect most of the centroids and the speed-up over exhaustive assignment falls to 1.04 to 1.16 on a 1M $\times$ 2,048 shape, with unchanged error (Appendix D). On real data the speed-up did not fade on any dataset we ran (smallest per-dataset geometric mean 3.09), but only two of them have 2,048 dimensions or more.
- The gain is smallest at $k = 8{,}192$. The cost model predicts none at k of about 2,000 or less, where a point would probe most cells; this was not measured.
- Ladder placement is worth about 4% of the geometric-mean speed-up, and the two top steps of the ladder exceed the interpolation limit on several datasets, which affects only the five-epoch column and gap(5). At the longest budgets e-means reaches a slightly lower error than any point of o-means' ladder on four test datasets.
- Each number is a mean over five seeds, or three at 10 million points, where the e-means paper uses ten. Nothing above 10 million points or 65,536 centroids was evaluated, while the e-means paper reaches billions of points. o-means has no CPU, multi-GPU, out-of-core or spherical mode.

7 Related work

Lloyd's algorithm (Lloyd, 1982) and the sequential rule of MacQueen (1967) are the two classical forms of k-means. Bottou and Bengio (1995) related both to gradient descent and to Newton's method, and Sculley (2010) introduced the mini-batch form with per-centroid step sizes. Later

mini-batch methods reduce the variance or the staleness of the updates (Newling and Fleuret, 2016; Zhu et al., 2024). Like e-means, o-means starts from random data points and not from a k-means++ seeding (Arthur and Vassilvitskii, 2007).

The exponential-moving-average update of a codebook was introduced with VQ-VAE (van den Oord et al., 2017) and used at scale in VQ-VAE-2 (Razavi et al., 2019). Codebooks trained this way suffer from unused codes, and restarting them is common practice in that literature, for example in Jukebox (Dhariwal et al., 2020). e-means (Bessa et al., 2026) specialised this family to the static k-means problem with a cosine step, revival by splitting, a closing mean and fitted laws for the batch size and step.

A large body of work makes the assignment step cheaper without changing its result. The methods of Elkan (2003), Hamerly (2010) and Ding et al. (2015) maintain bounds on point-to-centroid distances and use the triangle inequality to skip evaluations; Yinyang k-means groups the centroids, as we do, but to maintain one bound per group. These methods keep state per point across iterations and gain most when centroids move little, which is the opposite of the first passes of an EMA mini-batch run with revival, and the e-means authors note that such pruning parallelises poorly on GPUs. SuperKMeans (Kuffo et al., 2026) prunes within a distance evaluation, by dimensions.

Approximate assignment through a structure over the centroids has a long history in the construction of large visual vocabularies. Nistér and Stewénius (2006) build a tree by hierarchical k-means. Philbin et al. (2007) run Lloyd iterations in which the nearest centroid is found approximately with randomised k-d trees over the centroids, rebuilt at every iteration. Wang et al. (2012) compare each point only with nearby clusters. The inverted file (Sivic and Zisserman, 2003; Jégou et al., 2011) is the same two-level structure used for search, and faiss (Johnson et al., 2021; Douze et al., 2025) provides it on GPUs. e-means itself includes an inverted-file assignment for its sharded multi-GPU setting, with an index rebuilt at every batch, and its authors report a gain from $k = 2^{21}$ upward. o-means applies the idea at $k = 2^{13}$ to 2^{16} on one GPU, with a cell-sorted centroid table scanned in tensor-core tiles, a rebuild schedule tied to revival, and split pointers. Flash-KMeans (Yang et al., 2026) and faiss are the exact GPU implementations we compare with.

8 Conclusion

e-means reduced the number of passes that k-means needs at large k. o-means reduces the cost of a pass by comparing each point with the centroids of a few cells, and keeps the cells valid while the centroids are being trained. On eight held-out datasets and one RTX 5090 it reaches the error of e-means' one- to three-epoch runs in a geometric mean of 1/4.49 of the time, part of which is due to kernels and the larger part to the algorithm, and its error at equal time is not worse. In a comparison that was not pre-registered, it reaches the error of 25 Lloyd iterations with less than half the distance computations of a single Lloyd iteration. The method is approximate, was measured on one GPU model, and gains least at the smallest k we tested. Measurements on H100-class hardware against e-means' fastest kernels, a cell build that bounds cell sizes, and values of k above 65,536 remain open.

Reproducibility statement

The implementation is released as the `contrib/o_means` package of the Owlgebra AI fork of faiss, next to e-means' `contrib/e_means`, and takes the same arguments: `Omeans(d, k, nepochs=E)`. The package is a port of the research implementation that was measured here; with PyTorch's

deterministic algorithms enabled it returns the same centroids bitwise. It includes a script that runs o-means and e-means on one dataset and prints time and error. The numbers in this paper were produced with a separate evaluation harness whose protocol and scoring code were locked by hash before any scored run (Appendix A); every protocol run, including the single release run on the test suite, is a line in an append-only ledger. The data behind every table and figure are published alongside the release report and the protocol. The software versions were PyTorch 2.14.1 with CUDA 13.0, Triton 3.8.0, Flash-KMeans 0.3.1 and faiss-gpu 1.14.3; the e-means reference is its released code at faiss commit `7767c63`.

References

David Arthur and Sergei Vassilvitskii. k-means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.

Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 2055–2063, 2016.

Swann Bessa, Pierre-Emmanuel Mazaré, Gergely Szilvasy, Hugo Touvron, David Picard, Matthijs Douze, and Hervé Jégou. e-means: outperforming k-means (in one-tenth the epochs). *arXiv preprint arXiv:2611.09501*, 2026.

Léon Bottou and Yoshua Bengio. Convergence properties of the K-means algorithms. In *Advances in Neural Information Processing Systems 7*, pages 585–592, 1995.

Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: a generative model for music. *arXiv preprint arXiv:2005.00341*, 2020.

Yufei Ding, Yue Zhao, Xipeng Shen, Madanlal Musuvathi, and Todd Mytkowicz. Yinyang K-means: a drop-in replacement of the classic K-means with consistent speedup. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 579–587, 2015.

Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *IEEE Transactions on Big Data*, 2025. doi: 10.1109/TBDATA.2025.3618474.

Charles Elkan. Using the triangle inequality to accelerate k-means. In *Proceedings of the Twentieth International Conference on Machine Learning*, pages 147–153, 2003.

Greg Hamerly. Making k-means even faster. In *Proceedings of the 2010 SIAM International Conference on Data Mining*, pages 130–140, 2010.

Elias Jääsaari, Ville Hyvönen, Matteo Ceccarello, Teemu Roos, and Martin Aumüller. VIBE: vector index benchmark for embeddings. *arXiv preprint arXiv:2505.17810*, 2025.

Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

Leonardo Kuffo, Sven Hepkema, and Peter Boncz. A super fast k-means for indexing vector embeddings. *arXiv preprint arXiv:2603.20009*, 2026.

Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. University of California Press, 1967.

James Newling and François Fleuret. Nested mini-batch K-means. In *Advances in Neural Information Processing Systems 29*, pages 1352–1360, 2016.

David Nistér and Henrik Stewénius. Scalable recognition with a vocabulary tree. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, volume 2, pages 2161–2168, 2006.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, 2019.

James Philbin, Ondřej Chum, Michael Isard, Josef Sivic, and Andrew Zisserman. Object retrieval with large vocabularies and fast spatial matching. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2007.

Ali Razavi, Aäron van den Oord, and Oriol Vinyals. Generating diverse high-fidelity images with VQ-VAE-2. In *Advances in Neural Information Processing Systems 32*, 2019.

D. Sculley. Web-scale k-means clustering. In *Proceedings of the 19th International Conference on World Wide Web*, pages 1177–1178, 2010.

Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, volume 176 of *Proceedings of Machine Learning Research*, pages 177–189, 2022.

Josef Sivic and Andrew Zisserman. Video Google: a text retrieval approach to object matching in videos. In *Proceedings of the Ninth IEEE International Conference on Computer Vision*, pages 1470–1477, 2003.

Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

Aäron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems 30*, 2017.

Jing Wang, Jingdong Wang, Qifa Ke, Gang Zeng, and Shipeng Li. Fast approximate k-means via cluster closures. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 3037–3044, 2012.

Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Xiaoze Fan, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Kurt Keutzer, Song Han, Chenfeng Xu, and Ion Stoica. Flash-KMeans: fast and memory-efficient exact K-means. *arXiv preprint arXiv:2603.09229*, 2026.

Xueying Zhu, Jie Sun, Zhenhao He, Jiantong Jiang, and Zeke Wang. Staleness-reduction mini-batch K-means. *IEEE Transactions on Neural Networks and Learning Systems*, 35(10):14424–14436, 2024.

A Protocol details

This appendix completes Section 4. Together they contain the content of the locked evaluation protocol.

Data. The training rows of DEEP, SIFT and SPACEV are the first n rows of the published files; SIFT is divided by 512. ImageNet-CLIP uses the fixed one-million subset of the e-means benchmark. For the other VIBE sets, 20,000 rows drawn with a fixed seed are held out and the remainder is the training set; GloVe-200, CodeSearchNet, Landmark-DINO and CelebA are L2-normalised. The evaluation rows are the 10,000 rows that follow the training prefix for DEEP and SIFT, and 20,000 held-out rows otherwise. They are permuted with a fixed seed and cut into a validation half and a test half, as in the e-means benchmark. Each dataset has a recorded fingerprint of both halves and of a sample of training rows. Datasets whose host was unreachable were dropped from a suite before the lock and never after.

Suites. A smoke suite (DEEP 100k, $k = 4{,}096$) checks the plumbing and is never reported. dev-fast has five datasets with seeds 10 to 12. dev-full adds DEEP 10M, SIFT 10M, CC-News and CelebA and uses seeds 10 to 14 at $n \leq 2$ million and 10 to 12 at 10 million; the test suite uses the same seeds. Before the lock, CelebA was moved from the test suite to development and two 768-dimensional sets were added to development, so that development covers d from 96 to 2,048 while the test suite is mostly at $d \geq 512$.

Candidate rules. A candidate exposes one function that maps (data, k, starting centroids, effort, seed) to centroids, and a ladder of at most ten efforts in increasing cost. Its hyper-parameters may depend only on (n, d, k, effort) and on statistics computed inside the timed call. It may not branch on the identity of a dataset, read evaluation rows, or carry state between calls; the warm-up call may make the timed call cheaper only through compiled-kernel caches. Any precision and any library are allowed. The total time of the ladder may not exceed six times the reference's five-epoch time, otherwise the harness truncates the ladder.

Measurement. Before a timed call the GPU memory of the warm-up call is freed, and the call is bracketed by device synchronisation. Timed runs are serialised on the machine by a lock. A run is void and repeated if another process is using the GPU or if the one-minute load average exceeds 8 at its start. Per fit the harness records the held-out error, the training error by exact assignment on 200,000 fixed training rows, the number of dead clusters (fewer than 10 points), the imbalance $k \sum_j z_j^2 / n^2$ with z_j the final size of cluster j, the peak GPU memory, a check for non-finite values, and the distance evaluations and passes the candidate reports.

Pooling and envelope. A ladder point enters the pooled curve of a dataset only if every seed kept it. A target counts as reached when $\text{env}(t) \leq \bar{m}_e(T)\,(1 + 10^{-9})$; the tolerance absorbs the non-associativity of floating-point sums on the GPU, which makes identical e-means fits differ by up to 1.2×10^{-12} in relative error. A target that is never reached is assigned the speed-up $\min(0.5,\ 0.5\,\bar{t}_e(T)/t_{\max})$, with $t_{\max}$ the time of the most expensive ladder point. In Eq. (9), before the cheapest ladder point the envelope takes the error of the starting centroids.

Decision rules. The keep threshold during development is an improvement of the geometric-mean speed-up on dev-fast by more than $\max(2\varepsilon, 3\%)$, which is 3% with the measured ε of Appendix E.

Tuning, the fitting of rules and the analysis of failures used the development suites only, and per-run outputs of the test suite are read only through the aggregate. The test suite is run for release candidates only, and every such run is recorded.

Guardrails. Besides the three criteria of Section 4.5, each round reports the mean of gap(T) over datasets and $T \in \{2, 3, 5\}$, the gap at five epochs on the training error, the number of dead clusters, the peak memory against a limit of 30 GB, and the count of non-finite outputs. $T = 1$ is excluded from the mean gap because a ladder whose first point is slightly slower than e-means' one-epoch run would be charged the error of the starting centroids.

Reference files and the lock. The scoring code, the protocol text and the reference runs are identified by a hash, first recorded on 12 November 2026 at 13:53 UTC. The reference runs (the Lloyd@25 anchors and an e-means reference curve per dataset and seed) could not all be generated before the first lock because of machine failures; the missing ones, which include those of the test suite, were added in one recorded update at 15:50 UTC that changed no scoring code and no threshold. These files feed only the anchor lines of the figures and a guard against runaway ladders. The paired e-means runs that define the targets are produced inside each protocol run.

Deviations from the e-means paper's protocol.

- Hardware: one RTX 5090 in place of an H100, so the reference runs its Triton backend.
- One process per (dataset, seed) with paired and interleaved fits, in place of one process per fit.
- Three to five seeds in place of ten, pooled per dataset before the crossing is computed.
- Targets are e-means' error at 1, 2 and 3 epochs in place of the error of Flash-KMeans at 25 iterations, which is kept as an anchor. The e-means paper likewise measures time to quality only at the Lloyd@25 level, which e-means reaches at about two epochs, and reports errors at fixed budgets beyond that.
- The crossing is interpolated on the pooled ladder; the e-means paper's strict crossing is reported as a secondary value.
- The starting centroids follow the recipe of the e-means benchmark but do not reproduce its exact permutations on this GPU; the reproduction of the published results was validated at the level of medians.
- No configuration with 100 million or more points, for lack of memory.

The protocol also defines a secondary CPU track for candidates that ship a CPU implementation. o-means does not, and the track was not run.

B Development suite

Table 7 gives the results on the nine development datasets, scored on their validation halves. These datasets were used for tuning and are reported for completeness. The geometric-mean speed-up is 4.61 (4.19 strict) over 41 runs, the smallest speed-up of a target is 2.46 (CelebA, $T = 3$), all 123 per-seed comparisons favour o-means, and the peak memory was 8.6 GB.

Table 7. Development suite (nine datasets, validation halves). Columns as in Table 4.

			Speed-up at T epochs				Error at T=5	Distance ratio at T epochs		
Dataset	n	k	1	2	3	5[†]	(%)	1	2	3
DEEP 1M	1.00M	16,384	4.21	4.61	3.84	5.83	−0.15	0.13	0.13	0.18
SIFT 1M	1.00M	16,384	3.84	3.98	3.91	3.41	−0.10	0.14	0.17	0.14
GloVe-200	1.17M	16,384	4.83	4.66	4.56	2.94	−0.23	0.12	0.17	0.20
arXiv-768	1.32M	16,384	5.26	4.96	4.99	4.72	−0.22	0.11	0.13	0.14
Landmark-Nomic	0.74M	16,384	4.19	4.30	4.18	4.78	−0.31	0.12	0.14	0.15
DEEP 10M	10.0M	65,536	8.20	9.65	6.06	7.88	−0.16	0.06	0.05	0.09
SIFT 10M	10.0M	65,536	7.09	7.14	6.86	6.44	−0.06	0.06	0.06	0.06
CC-News	0.48M	8,192	4.53	2.70	3.07	2.99	−0.37	0.20	0.25	0.27
CelebA	0.18M	8,192	2.49	4.81	2.46	2.81	−0.14	0.22	0.18	0.23
Geometric mean			4.70	4.90	4.25	4.35		0.12	0.13	0.15

[†] Not part of the pre-registered criteria (unscored).

C Development rounds

Table 8 lists every round that was scored by the protocol: the rounds on dev-fast in the order in which they were run, the three rounds on dev-full (the closing mean with exhaustive assignment, the control v0, and the release candidate), and the two runs on the test suite. Rounds 42 and 43 are the same algorithm; the second was run from the merged code and returned bitwise-identical centroids.

The schedule of Section 3.2 was selected before round 40 by measuring, on single seeds, the miss rate and the training error against exhaustive assignment. Starting each run with one eighth or one quarter of a pass of exhaustive assignment did not help: on DEEP 1M a quarter-pass exhaustive start under the first rebuild schedule ended 0.27% above exhaustive assignment at one epoch, against 0.07% for the adopted schedule, and was slower. Doubling the number of probes throughout lowered the error by 0.03 to 0.08% for 40% more time. Refreshing the cells every ten batches of the first pass in place of every five saved 1 to 8% of the time and raised the error by 0.03 to 0.11 percentage points. Reducing p from 16 to 12 at $d < 512$ saved 8 to 14% of the time for 0.02 to 0.08 percentage points of error; by the measured exchange rate between error and time this is worth about 2% in the geometric mean, below the 3% threshold, and it was not adopted. Without split pointers the miss rate under the first rebuild schedule was 29 to 35% in place of 10 to 28%.

The adaptive variant of Section 5.4 selects 24 to 64 probes on GloVe-200 and leaves p unchanged on the four other dev-fast datasets, where its cost comes from the host reads at the points where the miss rate is sampled. A smaller instance of the ladder effect occurred in rounds 41 and 42: dropping the four-epoch point lowered the smallest target speed-up from 3.37 to 3.28, and restoring it in place of the 1.25-epoch point raised the geometric mean from 4.48 to 4.54.

D Synthetic shape study

Before the release, the final code was run on synthetic data in every distinct shape (n, d, k) of the 17 datasets and in two shapes outside them, 1M × 1,024 and 1M × 2,048. The purpose was to exercise the shapes of the test suite without touching its data. The data are Gaussian mixtures with 2,048 modes, L2-normalised where the corresponding real dataset is. There were 39 fits, at efforts 0.8, 2 and 12.8, and also 1.0 at 10 million points. The comparison is with the variant of round 2 in Table 8, which uses exhaustive assignment; we call it the exhaustive variant.

Table 8. Rounds scored by the protocol. The speed-up is the geometric mean over datasets and $T = 1, 2, 3$ relative to e-means; the minimum is the lowest speed-up of a single target. A round was kept if it improved the geometric mean on dev-fast by more than 3% without a guardrail regressing.

Round	Variant	Suite	Speed-up*	Smallest	Kept
0	e-means' algorithm on our kernels (v0)	dev-fast	1.51	1.22	yes
1	+ recency-weighted closing mean, first ladder	dev-fast	1.60	0.69	no (guardrail)
2	+ recency-weighted closing mean	dev-fast	1.66	1.27	yes
3	+ exact re-assignment of the last rows	dev-fast	1.60	1.27	no
40	two-level assignment, e-means' closing mean	dev-fast	4.29	3.37	yes
41	+ recency-weighted closing mean below two epochs, ladder without 4	dev-fast	4.48	3.28	no (smallest speed-up)
42	+ recency-weighted closing mean below two epochs, ladder adjusted	dev-fast	4.54	3.78	yes
43	the same, re-run from the merged code	dev-fast	4.44	3.74	number of record
4	+ recency-weighted closing mean (round 2), full suite	dev-full	1.74	1.22	confirmed
5	v0 on the full development suite	dev-full	1.53	1.21	decomposition only
44	release candidate on the full development suite	dev-full	4.61	2.46	confirmed
50	release candidate, the single held-out test run	test	4.49	2.54	success
51	v0 on the held-out test suite	test	1.43	1.19	decomposition only

* Speed-up in time to e-means' error, geometric mean over datasets and $T = 1, 2, 3$.

All 39 fits returned k finite centroids. The largest peak memory was 14.6 GB, at 1M $\times$ 2,048. The training error relative to the exhaustive variant at equal effort was between -0.77% and $+0.045\%$. At 10 million points the share of points missing their nearest centroid was 0.004 to 0.012% at efforts 0.8 and 1.0, 0.24 to 0.51% at effort 2 and 0.40 to 1.06% at effort 12.8, with 2.3 to 3.6% during passes 1 to 5 of the longest run at $d = 128$.

The time relative to the exhaustive variant depended strongly on the shape: 2.75 to 3.72 times faster at 10 million points and $d \leq 128$, 1.85 to 2.62 at 1M $\times$ 100, 3.1 to 3.95 at one million or more points and $d = 512$ or 768, 1.6 to 2.3 at 475k $\times$ 768, 1.67 to 1.76 at $d = 1{,}024$, 1.37 to 1.68 at the two shapes with $k = 8{,}192$ and $d \geq 2{,}048$, and 1.04 to 1.16 at 1M $\times$ 2,048. The slow shapes share a cause. On well-separated mixtures in high dimension, the Lloyd iterations on the centroids move a few cell centres toward the mean of many modes. Such a centre is nearer to every mode than the modes are to each other, and its cell absorbs most of the centroids: the largest cell held 1,900 to 4,600 centroids against a mean of 128, and points scanned 45 to 94% of the table. The miss rate in these cases is zero, so on these synthetic shapes the fit is only slower. Skipping the Lloyd iterations of the build did not help, since centroids themselves can be hubs. A build that bounds the cell size would address it and has not been implemented.

The same study measured the time ratio between neighbouring ladder points. Below five epochs it was at most 1.53. The two top steps, 5 to 8 and 8 to 12.8 epochs, were between 1.47 and 1.92 and thus often above the interpolation limit of 1.6. On the real test datasets these two steps are

between 1.45 and 1.73.

One more synthetic result comes from outside this study. The benchmark script shipped with the package uses by default a mixture of 65,536 Gaussian modes in 96 dimensions, one million points and $k = 16{,}384$, so there are more modes than clusters. On that data, with the default 16 probes, o-means' error at equal epochs is 1.2 to 1.7% above e-means' between two and five epochs. At three epochs, probing all 128 cells reduces the gap from 1.65% to 0.23%, so most of it is due to missed centroids. At equal time o-means' error is still lower than e-means'. These numbers were measured with that script and can be reproduced with it; they are not protocol results.

E Noise floor

All controls in this appendix were run on dev-fast, with three seeds, through the unmodified harness, before the lock.

Identical control. e-means as its own candidate, at $T \in \{1, 2, 3, 5\}$, measures the timing noise of the harness. Two rounds gave geometric-mean speed-ups of 1.00014 and 1.00012, with individual targets between 0.9985 and 1.0033.

Independent control. e-means with a different internal seed for its shuffles and revival, from the same starting centroids, is statistically equivalent to the reference but independent of it. On the ladder $\{0.8, 1, 1.25, 1.6, 2, 2.5, 4, 5, 6.25\}$, which has no point near three epochs, two internal seeds gave 0.966 and 0.953, with three-epoch targets between 0.75 and 1.01. The difference between the two, 0.013, is the tolerance ε; a timing replicate of the first reproduced it to within 0.0002. With a ladder that includes three epochs the same control gave 0.991. The crossing rule is therefore biased against the candidate by 1 to 5%, more where the ladder is sparse, which is why the protocol asks candidates to place ladder points near the cost of each target. The largest $|\mathrm{gap}(5)|$ of the independent control over datasets and rounds, 0.16% on DEEP 1M, is the tolerance $\varepsilon_{\mathrm{gap}}$.

Sensitivity controls. e-means slowed by a sleep of half its own run time, which is 1.5 times slower at identical quality, measured 0.665 against an expected 0.667. e-means run at 0.8 times the labelled number of epochs, which is faster and worse, measured 0.78.

Instability at five epochs. The seed-to-seed spread of e-means' error at fixed T is 0.05 to 0.34% on dev-fast, which exceeds what epochs beyond the fourth buy. For the independent control, the per-seed speed-up at the five-epoch target was about 0.43 on the 5 of 15 (dataset, seed) pairs that never reached the target and ranged from 0.83 to 2.12 on the others. On DEEP 1M the error of e-means is 0.38525 at five epochs and 0.38538 at eight, so longer runs do not close such a gap.

F Implementation notes and measurements

The measurements in this appendix were made during development on single seeds. They support statements in the main text and are not protocol results.

Kernels. Data whose largest magnitude lies outside $[2^{-8}, 2^{15}]$ are scaled by a power of two before the conversion to FP16, which is exact. In the two-level assignment, the first kernel scores a block of points against all cell centres on tensor cores, selects the p best cells of each point, and appends

each (point, cell) pair to the bucket of its cell, reserving slots with atomic counters so that the pairs end up grouped by cell without a sort. The second runs one program per tile of 64 pairs of one cell and multiplies the tile against that cell's contiguous block of centroids, keeping a running minimum. The third reduces the p partial results of each point, performs the split-pointer check and clears the counters. The update of Eq. (2), the refresh of the FP16 centroid table and its norms, and the accumulation of the closing sums are fused in one kernel. Revival runs with fixed launch shapes and reads the number of splits on the device. The only value read back to the host during a fit is the range check of the initial conversion. A run is bitwise reproducible for a given input, seed and GPU when the cell build is made deterministic.

Where e-means spends its time. With the benchmark's own call at $T = 2$, a fit on DEEP 1M ($d = 96$) takes 71 ms, of which 80.5% is the assignment kernel; on arXiv-768 it takes 454 ms, of which 86.3% is the assignment kernel. The per-batch conversion and gathering of rows accounts for 4.3% and 9.1%. Accumulation, the EMA update and revival together account for less than 5%. Inside the fit the assignment kernel runs at 104 and 170 TFLOPS on the two datasets.

Where o-means spends its time. A one-epoch o-means fit on DEEP 1M takes 8.8 ms: 4.35 in the two-level assignment, 2.21 in the cell builds, 0.77 in revival, 0.49 in accumulation, 0.48 in the initial conversion and 0.36 in the update. A two-epoch fit on arXiv-768 takes 82.8 ms: 50.5 in the assignment, 11.5 in accumulation, 9.8 in the update and 4.9 in the builds.

Two-level against exhaustive assignment at equal epochs. With the same seed, starting centroids and number of passes, and e-means' last-pass mean in both, the fit with two-level assignment was 1.96, 2.30 and 2.60 times faster than the fit with exhaustive assignment on DEEP 1M at 1, 2 and 3 epochs, and 3.12, 3.89 and 3.99 times on arXiv-768; on GloVe-200 and Landmark-Nomic-768 the factors at 1 and 2 epochs were 2.54 and 3.09, and 2.54 and 3.25. The training error was higher by 0.02 to 0.13% across these ten fits. The effect of the closing mean of Section 3.3 was the same on fits with two-level and with exhaustive assignment to within 0.02%.